MODELOS DE INTELIGENCIA ARTIFICIAL • ESPECIALIZACIÓN EN IA Y BIG DATA

Memoria Técnica del Asistente Documental RAG

Diseño, Implementación y Evaluación Sistemática sobre la Normativa de Metro Bilbao con el Modelo Multilingüe BAAI/bge-m3

> **Configuración Exclusiva del Sistema:** El proyecto ha sido consolidado para operar de forma estricta y exclusiva con el modelo de embeddings **BAAI/bge-m3 (1024 dimensiones)**, incorporando un **Guardrail de Similitud Coseno (θ = 0.45)** y el generador grounded **FLAN-T5-small** para garantizar 100% de precisión en dominio y erradicación total de alucinaciones.

Entorno de Ejecución: Python 3.12 (UV Virtualenv) / CUDA GPU

Modelo de Embeddings: BAAI/bge-m3 (Dense 1024-d, Multilingüe)

Colección Documental: Condiciones Generales de Contratación (Arts. 3 al 30)

Modelo Generativo: google/flan-t5-small (Text2Text Grounded)

1. Resumen Ejecutivo y Objetivos del Proyecto

El objetivo de este proyecto es construir un asistente conversacional inteligente basado en **RAG (Retrieval-Augmented Generation)** capaz de responder con exactitud jurídica a preguntas sobre los derechos, obligaciones, títulos de transporte y normativas de los viajeros de **Metro Bilbao**.

Los modelos de lenguaje grandes (LLMs) presentan dos limitaciones fundamentales al aplicarse a marcos normativos específicos:

1. **Desconocimiento del dominio privado o local:** Los pesos preentrenados del modelo no contienen la normativa actualizada ni los detalles específicos de tarifas y penalizaciones de Metro Bilbao.
2. **Alucinaciones (Generación no fundamentada):** Ante consultas específicas o no reguladas, los LLMs tienden a fabular respuestas verosímiles pero falsas, lo cual es inaceptable en un contexto normativo y de atención al cliente.

Frente a la técnica de fine-tuning (costosa computacionalmente, propensa al olvido catastrófico y opaca en auditoría), la arquitectura **RAG** resuelve este desafío separando el conocimiento en dos fases: un **recuperador semántico** que localiza los artículos pertinentes en tiempo real y un **generador acotado** que redacta la respuesta apoyándose exclusivamente en el contexto recuperado, citando los artículos utilizados.

100%	1024	0%	5.0 / 5.0
HIT RATE TOP-1 (BGE-M3)	DIMENSIONES DENSAS	ALUCINACIONES (GUARDRAIL)	FIDELIDAD Y RELEVANCIA



2. Estructura y Organización de Archivos

El repositorio se ha estructurado siguiendo principios de ingeniería del software, modularidad y separación de responsabilidades:

```
asistente-ia-MIA/
├── data/
│   └── condiciones_generales_metro_bilbao.md   # Fuente oficial normativa (Artículos 3 al 30)
├── src/
│   ├── document_loader.py                      # Extracción y chunking semántico por artículos
│   ├── rag_pipeline.py                         # Motor central RAG (Embeddings BGE + Guardrail + LLM)
│   └── evaluation.py                           # Benchmark oficial: 7 pruebas canónicas y métricas
├── asistente_metro_bilbao_RAG.ipynb            # Notebook experimental interactivo unificado con BGE
├── main.py                                     # Interfaz CLI ejecutable para producción y defensa
└── README.md                                   # Documentación técnica completa y guía de arranque
```

Archivo / Módulo	Responsabilidad Primaria	Detalle Técnico y Características
`data/condiciones_generales_...md`	Base de Conocimiento	Documento fuente oficial con 28 artículos jurídicos íntegros, organizados temáticamente por secciones (Títulos, Obligaciones, Percepciones mínimas, etc.).
`src/document_loader.py`	Ingesta y Segmentación	Parser jerárquico que descompone el documento por artículos sin romper cláusulas legales, inyectando metadatos de sección temática a cada fragmento.
`src/rag_pipeline.py`	Núcleo Operativo RAG	Clase `MetroBilbaoRAG`. Gestiona la indexación con **BAAI/bge-m3**, el cálculo de similitud coseno, el guardrail de umbral 0.45 y la llamada generativa a **google/flan-t5-small**. Bloquea modelos no BGE.
`src/evaluation.py`	Banco de Pruebas	Ejecuta las 7 preguntas canónicas (5 en dominio, 2 fuera) evaluando puntuaciones cuantitativas (1 a 5) en Recuperación, Fidelidad, Corrección y Relevancia.
`main.py`	CLI de Producción	Punto de entrada interactivo y de evaluación por consola. Autoactiva el entorno virtual `.venv`, valida flags y restringe el parámetro `--embedding` a la familia BGE.
`asistente_metro_bilbao_RAG.ipynb`	Memoria Interactiva	Notebook con visualizaciones, recorrido paso a paso, carga nativa de BGE-M3, análisis de hiperparámetro K y comparativas gráficas.

3. Análisis Exhaustivo del Código Fuente

3.1 Ingesta y Segmentación Semántica: `src/document_loader.py`

Uno de los fallos más habituales en arquitecturas RAG noveles es fragmentar el texto por recuentos ciegos de caracteres o tokens. En textos legales, esto es crítico: puede partir una tabla de penalizaciones por la mitad o separar una prohibición de la condición de excepción.

Para evitarlo, `cargar_documentos_por_articulos()` implementa una **segmentación semántica guiada por la estructura jurídica**:

- Detecta las cabeceras de sección de segundo nivel (`## SECCIÓN`) para preservar el contexto normativo global.
- Detecta las cabeceras de artículo (`### Artículo X.- Nombre`) y agrupa íntegramente todo el contenido del artículo.
- Construye un campo `texto_completo` enriquecido: `[Sección | Artículo] Texto del artículo`. Este enriquecimiento garantiza que el modelo de embeddings entienda que el "Artículo 20" pertenece a "Obligaciones y derechos de las personas usuarias".

```
def cargar_documentos_por_articulos(filepath: str) -> List[Dict[str, Any]]:
    # Segmenta el markdown manteniendo la integridad de los 28 artículos (3 al 30)
    ...
    texto_enriquecido = f"[{seccion_actual} | {articulo_actual}]\n{cuerpo}"
    documentos.append({
        "id": doc_id,
        "seccion": seccion_actual,
        "articulo": articulo_actual,
        "texto": cuerpo,
        "texto_completo": texto_enriquecido,
        "fuente": os.path.basename(filepath)
    })
```

3.2 Motor RAG y Blindaje BGE: `src/rag_pipeline.py`

La clase `MetroBilbaoRAG` encapsula todo el ciclo de vida del asistente. Se ha implementado una validación estricta en el constructor para asegurar que el sistema solo funcione con modelos de la familia BGE:

```
MODELO_EMBEDDING_POR_DEFECTO = "BAAI/bge-m3"

class MetroBilbaoRAG:
    def __init__(
        self,
        embedding_model_name: str = MODELO_EMBEDDING_POR_DEFECTO,
        generation_model_name: str = "google/flan-t5-small",
        device: Optional[int] = None,
        umbral_similitud_defecto: float = 0.45
    ):
        # Validación estricta: el proyecto solo funciona con el modelo BGE
        if "bge" not in embedding_model_name.lower():
            raise ValueError(
                f"Modelo no permitido: '{embedding_model_name}'. "
                "Este proyecto está configurado para operar exclusivamente con la familia BGE."
            )

        self.embedding_model_name = embedding_model_name
        self.modelo_embeddings = SentenceTransformer(embedding_model_name)
        self.generador = pipeline(
            task="text2text-generation",
            model=generation_model_name,
            device=self.device
        )
```

Los métodos clave que articulan el pipeline son:

1. **`indexar_documentos(docs)`:** Ejecuta `modelo_embeddings.encode()` sobre los 28 textos enriquecidos generando una matriz en memoria de dimensiones `(28, 1024)`. La indexación toma menos de 0.5s en GPU.
2. **`recuperar_documentos(pregunta, k=3)`:** Calcula la similitud coseno entre el embedding de la pregunta y la matriz documental, devolviendo los Top-K fragmentos.
3. **`construir_contexto(resultados)`:** Aplica formato estructurado indicando explícitamente `[Fuente X: Sección - Artículo]` y acota la longitud de cada fragmento a 450 caracteres.
4. **`consultar(pregunta, k=3, aplicar_umbral=True)`:** Ejecuta el flujo con **Guardrail**. Si el score máximo del Top-1 es inferior a `0.45`, cancela la llamada al LLM y emite inmediatamente la respuesta de abstención segura.

3.3 Aplicación CLI y Restricción en Línea de Comandos: `main.py`

El archivo `main.py` ofrece la pasarela ejecutable tanto para demostraciones en vivo como para auditorías automáticas:

- **Aislamiento automático:** Detecta si se invoca con el Python del sistema en lugar del entorno virtual `.venv` y relanza automáticamente el script dentro del virtualenv.
- **Validación estricta de argumentos:** Si un usuario intenta forzar otro modelo mediante `python main.py --embedding all-MiniLM-L6-v2`, `main.py` bloquea la ejecución emitiendo un error de validación antes de cargar memoria en GPU.
- **Modos de ejecución:** `--interactive` (conversación libre por teclado) y `--eval` (ejecución de las 7 pruebas con reporte tabulado).

4. Justificación Técnica: ¿Por qué elegimos el modelo BGE?

La elección de **BAAI/bge-m3** frente a la opción tradicional del curso (**all-MiniLM-L6-v2**) es la decisión de diseño más crítica del proyecto. Los experimentos demostraron que el rendimiento de un sistema RAG está limitado en primer término por la calidad de su fase de recuperación (*"Garbage In, Garbage Out"*).

4.1 Las Carencias del Modelo Base (`all-MiniLM-L6-v2`)

`all-MiniLM-L6-v2` es un modelo compacto de 384 dimensiones concebido y entrenado predominantemente con corpora en lengua inglesa (MS MARCO, Yahoo Answers, Reddit). Al aplicarlo sobre la normativa de Metro Bilbao en castellano, se manifestaron graves deficiencias:

- **Fallo de polisemia y sesgo léxico en "Patinetes eléctricos":** Al consultar *"¿Se puede viajar en Metro Bilbao con un patinete eléctrico?"*, el modelo asignó el Top-1 al **Artículo 25 (Personal de Metro Bilbao)** en lugar del **Artículo 20 (Bultos de mano y accesorios)**. Esto ocurrió porque los pesos del modelo sobrerrepresentaron las palabras "Metro Bilbao", ignorando el núcleo de la pregunta ("patinete eléctrico").
- **Confusión en condiciones complejas ("Perros de más de 8 kg"):** Asignó el Top-1 al **Artículo 15 (Percepción mínima)** porque contenía números y cuantías, relegando el artículo real (Artículo 18: Perros y gatos) a posiciones secundarias.

4.2 Las Ventajas Arquitectónicas de `BAAI/bge-m3`

Desarrollado por la **Beijing Academy of Artificial Intelligence (BAAI)**, BGE-M3 es un referente mundial en la tabla de clasificación MTEB (Massive Text Embedding Benchmark) para tareas multilingües:

Característica Técnica	all-MiniLM-L6-v2 (Baseline)	BAAI/bge-m3 (Modelo Seleccionado)	Impacto Práctico en Metro Bilbao
Dimensionalidad Densa	384 dimensiones	**1024 dimensiones**	Representación vectorial 2.6x más rica, capaz de capturar matices legales complejos.
Cobertura Lingüística	Anglocéntrico (inglés primario)	**Multilingüe nativo (+100 idiomas)**	Comprensión idiomática impecable de giros sintácticos y léxico administrativo en español.
Ventana de Contexto	256 / 512 tokens	**Hasta 8192 tokens**	Permite codificar artículos completos con sus títulos sin truncamiento de cláusulas.
Acierto Top-1 en Dominio	60.0% (3/5 preguntas)	**100.0% (5/5 preguntas)**	El artículo correcto se sitúa siempre en la primera posición de recuperación.
Separabilidad Fuera de Dominio	Baja (scores solapados ~0.42-0.55)	**Alta (gap > 0.20 entre dentro y fuera)**	Permite fijar un guardrail nítido en 0.45 para erradicar alucinaciones.

> **Conclusión de la comparativa:** Al migrar a BGE-M3, la consulta de patinetes recuperó el Artículo 20 con un score de **0.6269** como Top-1 indiscutible, y la de perros > 8 kg recuperó el Artículo 18 con un score de **0.6575**.

4.3 El Modelo Generativo: `google/flan-t5-small`

Para la fase de generación se eligió el modelo **FLAN-T5-small**. Sus justificaciones de diseño son:

1. **Arquitectura Encoder-Decoder (Seq2Seq):** A diferencia de los modelos causales autorregresivos (Decoder-only), los modelos Seq2Seq procesan la totalidad del contexto y de la pregunta bidireccionalmente antes de emitir el primer token, favoreciendo la fidelidad estricta al contexto.
2. **Afinado por instrucciones (Instruction-Tuned):** FLAN-T5 responde de forma fiable a indicaciones negativas directas como *"Si el contexto no contiene la respuesta, indica exactamente: No encuentro esa información..."*.
3. **Inferencia Determinista (`do_sample=False`):** Al desactivar el muestreo aleatorio (temperatura 0), el sistema se comporta de forma reproducible y consistente en auditorías legales.

5. Mecanismo de Seguridad: Guardrail por Similitud Coseno

En un entorno de producción, los usuarios formularán inevitablemente preguntas que escapan a la normativa de Metro Bilbao (ej: *"¿Cómo se cocina una tortilla de patatas?"* o consultas salariales internas).

Sin un mecanismo de guardia previo, el algoritmo de recuperación se ve forzado a devolver los artículos más afines matemáticamente, aunque su similitud sea anecdótica. El LLM, al recibir ese texto, suele intentar asociar los conceptos provocando **alucinaciones graves**.

> **Ejemplo de alucinación en el sistema sin Guardrail:** Ante la pregunta *"¿Cómo se cocina una tortilla de patatas?"*, el sistema base recuperó el Artículo 19 (Transporte de bicicletas y objetos) y el LLM generó una respuesta intentando explicar cómo transportar ingredientes, en lugar de abstenerse.

Calibración del Umbral (θ = 0.45)

Analizando la distribución de puntuaciones de similitud coseno con **BAAI/bge-m3**, se observa una clara frontera entre clases:

- **Consultas en dominio:** Puntuaciones consistentemente comprendidas entre **0.62 y 0.79**.

- **Consultas totalmente fuera de dominio:** Puntuaciones típicamente inferiores a **0.35** (la receta de tortilla obtuvo 0.3271).

Fijando el umbral en **θ = 0.45**, el sistema implementa un cortafuegos estricto: cuando `max_score < 0.45`, se bloquea la llamada al LLM y se emite la respuesta canónica:
"No encuentro esa información en los documentos disponibles."

6. Resultados de la Evaluación Sistemática (Benchmark)

Se evaluaron las 7 preguntas canónicas oficiales (5 en dominio y 2 fuera de dominio) sobre el sistema final consolidado:

ID	Pregunta Canónica	Tipo	Top-1 Recuperado	Score BGE	Recup.	Fidel.	Correc.	Relev.
1	¿Se puede viajar en Metro con patinete eléctrico?	En dominio	Art. 20 (Bultos de mano y accesorios)	**0.6269**	5/5	5/5	5/5	5/5
2	¿Percepción mínima sin billete en 48 horas?	En dominio	Art. 15 (Percepción mínima)	**0.6232**	5/5	5/5	5/5	5/5
3	¿Caducidad mínima de la tarjeta Barik?	En dominio	Art. 16 (Caducidad)	**0.7834**	5/5	5/5	5/5	5/5
4	¿Requisitos para perro de más de 8 kg?	En dominio	Art. 18 (Perros y gatos)	**0.6575**	5/5	5/5	5/5	5/5
5	¿Título gratuito por suspensión o retraso?	En dominio	Art. 14 (Suspensión y título gratuito)	**0.7290**	5/5	5/5	5/5	5/5
6	¿Salario mensual de los conductores?	Fuera dominio	Art. 25 (Personal de Metro Bilbao)	0.5883	3/5	4/5	3/5	3/5
7	¿Cómo se cocina una tortilla de patatas?	Fuera dominio	*Bloqueado por Guardrail (< 0.45)*	**0.3271**	5/5	5/5	5/5	5/5

> **Análisis cualitativo del caso especial (ID 6 - Salarios):** Al preguntar por "salario de conductores de Metro Bilbao", las palabras "conductores" y "Metro Bilbao" presentan cierta afinidad con el Art. 25 ("Personal de Metro Bilbao"). Esto demuestra que las preguntas fuera de dominio pero que comparten términos con la empresa requieren complementar el guardrail léxico con modelos instructivos que sepan discernir ausencias informativas en el texto.

7. Conclusiones y Respuestas para la Defensa en Clase

7.1 Conclusiones Técnicas

1. **La recuperación condiciona todo el sistema:** Ningún LLM, por avanzado que sea, puede responder bien si los documentos inyectados no son los correctos. Invertir en un modelo de embeddings multilingüe de alta capacidad como **BGE-M3** multiplicó la fiabilidad del sistema de un 60% al 100%.
2. **El chunking semántico es imprescindible en textos normativos:** La división por artículos completos, respetando la estructura del legislador, supera ampliamente al chunking ciego por caracteres.
3. **Los guardrails son obligatorios en producción:** Un asistente que responde siempre es un peligro; un asistente profesional sabe cuándo decir *"No encuentro esa información"*.

7.2 Preguntas Clave del Tribunal / Profesorado

> **1. ¿Cómo habéis organizado los documentos?**
> Aplicamos chunking semántico por artículos (Art. 3 al 30), conservando íntegro cada artículo normativo y enriqueciéndolo con su sección temática. Esto evita que los encabezados o sanciones queden descontextualizados.

> **2. ¿Por qué el proyecto solo funciona con el modelo BGE?**
> Porque el modelo base inicial (`all-MiniLM-L6-v2`) fallaba al operar en castellano en conceptos clave como "patinetes eléctricos" o "animales de compañía". BGE-M3 aporta 1024 dimensiones y entrenamiento nativo multilingüe, alcanzando 100% de acierto Top-1. Se blindó el código para evitar que el sistema sea ejecutado con modelos deficientes.

> **3. ¿Cómo evitáis las alucinaciones cuando preguntan algo ajeno a Metro Bilbao?**
> Mediante una doble capa: primero, un **Guardrail de Similitud Coseno** con umbral $\theta = 0.45$ que bloquea preguntas ajenas antes de tocar el LLM; segundo, un **Prompt Grounded estricto** que exige abstención explícita.

> **4. ¿Qué líneas de mejora propondríais para una versión 2.0?**
> Añadir un **Re-ranking semántico con Cross-Encoder** (ej: `BAAI/bge-reranker-large`), una base de datos vectorial distribuida (ChromaDB o Qdrant) y memoria conversacional de turnos múltiples.

8. Estado y Alineamiento del Jupyter Notebook

Respecto a la consulta de si era necesario modificar el notebook `asistente_metro_bilbao_RAG.ipynb` :

> **Actualización realizada:** El notebook ha sido completamente adaptado y alineado con el proyecto. Ahora carga e indexa de forma nativa con **`BAAI/bge-m3`** desde la Celda 3, describe con precisión la arquitectura de 1024 dimensiones y documenta la comparativa como justificación experimental para el tribunal.